**Guía de demostración en Flask**

**Preparación antes de exponer**

1. Descarga el paquete Laboratorio listo (ZIP) desde Descargar Recursos.
2. Haz clic derecho en el ZIP y elige Extraer todo. Entra en la carpeta laboratorio-flask descomprimida. No ejecutes archivos dentro del ZIP.
3. Necesitas Python 3.11 o posterior. Si no está instalado, obtén el instalador de https://www.python.org/downloads/ y habilita la opción de agregar Python al PATH. El paquete no instala Python ni cambia ajustes del sistema.
4. En Windows, ejecuta INICIAR.cmd con doble clic. Como alternativa, abre una terminal en esa carpeta y ejecuta py -3 main.py. En macOS o Linux, usa python3 main.py.
5. La primera ejecución crea .venv e instala las dependencias de requirements.txt desde PyPI. Necesita Internet. Las siguientes reutilizan el entorno si no cambian las dependencias.
6. El lanzador elige un puerto libre y abre automáticamente la página local. Mantén la terminal abierta. Si el navegador no se abre, copia la dirección que aparece en la terminal.
7. Todos los enlaces de prueba están en la portada del laboratorio. No es necesario cambiar direcciones a mano ni escribir el código. Solo se utilizan dos registros ficticios en memoria: ana y bruno.

El servidor escucha únicamente en 127.0.0.1. Los patrones vulnerables son deliberados para esta práctica. No despliegues el laboratorio en Internet. La web de las diapositivas solo distribuye los archivos y no ejecuta el backend vulnerable.

**Paso 1. Demostrar inyección SQL**

1. En la portada local, pulsa Búsqueda normal. Debe aparecer Registros: 1 y el usuario ana. Explica que esta es la respuesta legítima.
2. Vuelve a Inicio del laboratorio y pulsa Probar SQL vulnerable. Debe aparecer Registros: 2 con ana y bruno.
3. Señala la entrada de prueba mostrada en la página: ' OR '1'='1. Explica que el programa pegó esa entrada en una consulta y la condición terminó siendo verdadera para ambos registros.
4. Vuelve a la portada y pulsa Probar SQL protegido. Debe aparecer Registros: 0. La cadena completa se interpreta como un nombre, no como una condición SQL.
5. Si deseas mostrar el código, abre app.py en el editor y busca lookup. Compara la concatenación vulnerable con db.execute que usa ? y envía el valor por separado.

Frase para exponer: el problema no está en buscar un nombre; está en permitir que el nombre cambie la orden. La prevención es separar la consulta de sus valores. El ejemplo no borra registros, no implementa un login y no demuestra todas las variantes posibles de SQLi.

**Paso 2. Demostrar XSS**

1. Desde la portada pulsa Probar XSS vulnerable. El navegador debe mostrar una alerta que dice DEMO LOCAL. Ciérrala.
2. Explica que el saludo insertó contenido sin tratarlo como texto y el navegador ejecutó el script. La alerta no lee cookies ni envía información.
3. Vuelve al inicio y pulsa Probar XSS protegido. Ahora debe verse la cadena del script escrita en la página, sin una alerta.
4. Si muestras código, compara xss_bad y xss_good en app.py. La segunda función coloca el valor en una plantilla fija con escape HTML.

Frase para exponer: recibir texto no significa darle permiso para ejecutarse. Esta prueba muestra un caso de XSS reflejado en HTML; otros contextos de salida requieren sus propias protecciones. No utilices safe con entradas no confiables.

**Paso 3. Comprobar la protección CSRF**

1. Entra en Perfil / CSRF. Cambia el alias y pulsa Guardar con token. El nuevo alias debe aparecer como guardado.
2. Pulsa Enviar sin token (esperar 400). Debe mostrarse Solicitud rechazada / HTTP 400.
3. Pulsa Volver al perfil. Comprueba que se conserva el alias válido y no se guardó No debe guardarse.
4. Opcional: abre las herramientas del navegador con F12, entra en Elements, localiza profile-form y elimina únicamente su input oculto csrf_token. Envía ese formulario y observa el mismo rechazo. Recargar restaura el token.

Frase para exponer: una sesión válida identifica una continuidad, pero no prueba por sí sola la intención de modificar datos. Aquí comprobamos el rechazo de una petición sin token; no simulamos un ataque completo desde otro origen. La protección del servidor permanece activada.

**Paso 4. Observar la cookie de sesión**

1. Con el perfil abierto, entra en DevTools: Application o Almacenamiento, después Cookies y el origen local mostrado en la terminal.
2. Busca la cookie session y revisa HttpOnly y SameSite=Lax. No copies su contenido a capturas ni lo compartas.
3. Explica que Secure está desactivado solo porque este ejemplo utiliza HTTP en loopback. En una aplicación publicada con HTTPS se debe activar.
4. Recuerda que la sesión estándar de Flask está firmada, no cifrada. No contiene contraseñas ni información real en esta práctica.

**Terminar y repetir**

Regresa a la terminal y pulsa Ctrl+C. Se cierra el servidor. La base de datos solo existió en memoria y la clave de sesión se genera de nuevo al iniciar. Para repetir, ejecuta de nuevo INICIAR.cmd o main.py. El puerto puede cambiar; utiliza la dirección que imprima el lanzador, no un marcador de una sesión anterior.

Para comprobar resultados sin abrir una ventana: py -3 main.py --check. Debe terminar con un mensaje OK sobre SQL, XSS, CSRF y la restricción local. Para iniciar sin abrir el navegador: py -3 main.py --no-browser. Estas opciones sirven para ensayar; la demostración ante el público se realiza en el navegador.

**Si algo falla**

Python no encontrado: instala Python 3.11 o posterior y vuelve a abrir la terminal. Si py no existe pero python sí, usa python main.py. Una instalación incompleta de dependencias se vuelve a intentar al ejecutar el lanzador.

No se descargan bibliotecas: comprueba Internet y vuelve a ejecutar. No hace falta desactivar antivirus ni cambiar la política de PowerShell. El instalador de dependencias trabaja dentro de .venv.

No abre la página: copia exactamente la dirección de la terminal mientras el servidor está abierto. No mezcles localhost con 127.0.0.1 y no reutilices un puerto antiguo.

El formulario válido da 400: vuelve al perfil, permite las cookies locales y recarga. Al reiniciar el servidor cambia la clave de sesión y un formulario anterior puede tener un token inválido.

La alerta no aparece: utiliza el botón de prueba vulnerable de la portada. La ruta protegida debe mostrar el script como texto; no debería ejecutar la alerta.

**Diccionario de términos**

Flask: herramienta de Python para crear aplicaciones web.

Flask-WTF: biblioteca que proporciona, entre otras funciones, validación CSRF.

Entorno virtual o .venv: carpeta que contiene las dependencias aisladas del laboratorio.

PyPI: repositorio de paquetes de Python desde el que pip instala bibliotecas.

Loopback: conexión a la propia computadora, representada aquí por 127.0.0.1.

Puerto: número que distingue el servicio local. El lanzador pide uno disponible.

SQLi: entrada que cambia una instrucción SQL al mezclarse con ella.

XSS: contenido de una entrada que llega a ejecutarse en el navegador.

CSRF: solicitud inducida que intenta aprovechar credenciales enviadas automáticamente.

Token CSRF: valor que el servidor comprueba antes de aceptar una modificación.

Escape HTML: representación que permite mostrar caracteres como texto sin convertirlos en etiquetas.

Cookie firmada: dato cuya firma permite detectar cambios, sin ocultar necesariamente su contenido.

**Referencias técnicas**

Flask: https://flask.palletsprojects.com/en/stable/web-security/

OWASP Cheat Sheet Series: https://cheatsheetseries.owasp.org/

Flask-WTF: https://flask-wtf.readthedocs.io/en/stable/csrf/